

EJERCICIOS DE CADENAS DE CARACTERES

1. Realiza un programa que pida una frase y la muestre invertida, sin usar métodos específicos para ello.
2. Realiza un programa que pida una frase y cuente la cantidad de vocales que están acentuadas.
3. Realiza un programa que pida una dirección web, del estilo de: `http://www.google.com`, la analice y muestre esto en pantalla:

Protocolo: `http`

Host: `www`

Nombre de dominio: `google.com`

4. Realiza un programa que permita, a partir de una frase introducida por teclado, sustituir en ella una letra cualquiera por otra (ambas tendrán que ser pedidas por teclado).
5. Realiza un programa que, a partir de una palabra introducida por teclado, diga si es palíndromo o no. Una palabra es palíndromo si se lee de la misma forma en sus dos sentidos.
6. Realiza un programa que, a partir de una frase que hemos introducido por teclado, cuente cuantas palabras tiene.
7. Crea un array de números y otro de String de 10 posiciones donde insertaremos notas entre 0 y 10 (debemos controlar que se inserte una nota valida), pudiendo ser decimal la nota en el array de números, en el de Strings se insertaran los nombres de los alumnos. Después, crearemos otro array de String del mismo tamaño donde insertaremos el resultado de la nota con palabras, y eso lo tendrá que hacer automáticamente el programa según estos baremos:

Si la nota esta entre 0 y 4,99 , sera “Suspenso”.

Si esta entre 5 y 6,99 , será “Aprobado”.

Si esta entre 7 y 8,99 sera un “Notable”.

Si esta entre 9 y 10 sera un “Sobresaliente”.

Al final, recorre los vectores para mostrar por pantalla, el nombre del alumno, su nota y su calificación de forma cualitativa equivalente.

8. Realiza un programa que implemente el algoritmo de compresión RLE (Run Length Encoding). Básicamente el algoritmo reemplaza secuencias consecutivas de un carácter por el carácter unido a las repeticiones de éste. Actualmente se utiliza poco, pero antiguamente se usaba para comprimir fotos en blanco y negro. Veamos un ejemplo del algoritmo:

Entrada: `aaaabbccccccdd` (tamaño 15 bytes)

Salida: `a4b2c7d2` (tamaño 8 bytes)

ALGORITMO

Follow these steps to learn and understand the algorithm.

A) Create the output string which is empty and count variable which value is 0 for now.

- B) Take the first character and compare it with the next character. If matches, increment count by 1. For each successful match, the count will be incremented by 1. Then take the next character and do compare until it finds no match.
- C) Once you find the no match, add the current character and count to the output string.
- D) Take the next unmatched character and repeat steps B and C.
- F) After completing all characters finally, the output string will have the compressed string.
9. Programa para codificar o decodificar un texto utilizando el método de cifrado de César. Supondremos que el texto solo contiene letras mayúsculas o minúsculas. Las letras serán las correspondientes al alfabeto inglés (26 caracteres, excluimos la ñ y Ñ). En este método de cifrado cada letra del texto se sustituye por otra letra que se encuentra N posiciones adelante en el alfabeto. Se considera que el alfabeto es circular, es decir, la letra siguiente a la 'z' es la 'a'.

Por ejemplo, si N es 3, la 'a' se transformaría en 'd', la 'b' en 'e', la 'c' en 'f', etc.
Ejemplo de cifrado César: si el texto es "casa" y N = 3 el texto cifrado es "fdvd"

Para descifrar un texto se realiza la operación contraria. Se calcula la letra que está N posiciones por detrás en el alfabeto. Como el alfabeto es circular, la letra anterior a la 'a' es la 'z'.

El programa pedirá por teclado un texto, a continuación el valor de N y si queremos codificar o decodificar el texto. Finalmente se mostrará el texto resultante.

10. Dada una cadena de entrada, comprobar si es una contraseña FUERTE o DÉBIL. Se considera que una contraseña es fuerte si contiene 8 o más caracteres, y entre ellos al menos hay una mayúscula, una minúscula, un signo de puntuación y un dígito. Resuélvelo sin usar expresiones regulares, y luego con expresiones regulares.
11. Realiza un programa para validar un DNI. Para ello, pediremos el DNI por teclado y calcularemos la letra que le corresponde comparándola con la introducida. Para calcular la letra, cogeremos el resto de dividir nuestro DNI entre 23, el resultado debe estar entre 0 y 22. Haz un método donde según el resultado de la anterior fórmula busque en un array de caracteres la posición que corresponda a la letra. Esta es la tabla de caracteres:

RESTO	0	1	2	3	4	5	6	7	8	9	10	11
LETRA	T	R	W	A	G	M	Y	F	P	D	X	B

RESTO	12	13	14	15	16	17	18	19	20	21	22
LETRA	N	J	Z	S	Q	V	H	L	C	K	E

Cuando lo tengas, modifica el programa para que, si un usuario introduce un DNI con formato.